

Minicurso de C

Aula 05 - Strings, Struct e Depuração

Raíssa Cavalcante



Quem sou eu?

- Estudante de Ciência da Computação (6º período)
- Monitora de Programação C
- Diretora de Inovação na LAIA
- Pesquisadora PIBITI com a prof. Edilayne Meneses



Strings



Agradecimento a
Gustavo pelos slides!

Strings: Conceito de String em C

O termo *string* serve para identificar uma sequência de caracteres.

Na prática, as *strings* são usadas para representar textos, palavras, ...

Em C, ao contrário de outras linguagens, **não** existe um tipo de dados nativo para representação uma *string*.

Para representar uma *string* em C, devemos criar um vetor de caracteres, ou seja, um vetor do tipo *char*.

```
char nome[] = "Gustavo";
```

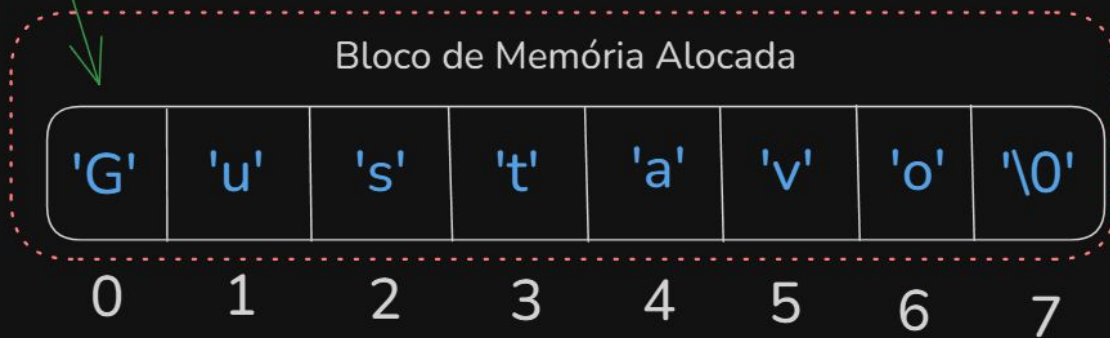
Strings: Conceito de String em C

Ocorre que o último caracter de uma string deve ser sempre o caracter nulo `'\0'` que serve para indicar o final da *string*.

```
char nome[] = "Gustavo";
```

Compilador

Bloco de Memória Alocada



Strings: Conceito de String em C

Na memória do computador, **não** é armazenado o caractere em si, mas sim um número que o representa. Esse número é definido por padrões como a **tabela ASCII**, que associa cada símbolo a um valor numérico específico.

'G'	'u'	's'	't'	'a'	'v'	'o'	'\0'
0	1	2	3	4	5	6	7
							
71	117	115	116	97	118	111	0
0	1	2	3	4	5	6	7

Strings: Tabela ASCII

ASCII control characters			ASCII printable characters					
00	NULL	(Null character)	32	space	64	@	96	`
01	SOH	(Start of Header)	33	!	65	A	97	a
02	STX	(Start of Text)	34	"	66	B	98	b
03	ETX	(End of Text)	35	#	67	C	99	c
04	EOT	(End of Trans.)	36	\$	68	D	100	d
05	ENQ	(Enquiry)	37	%	69	E	101	e
06	ACK	(Acknowledgement)	38	&	70	F	102	f
07	BEL	(Bell)	39	'	71	G	103	g
08	BS	(Backspace)	40	(	72	H	104	h
09	HT	(Horizontal Tab)	41	)	73	I	105	i
10	LF	(Line feed)	42	*	74	J	106	j
11	VT	(Vertical Tab)	43	+	75	K	107	k
12	FF	(Form feed)	44	,	76	L	108	l
13	CR	(Carriage return)	45	-	77	M	109	m
14	SO	(Shift Out)	46	.	78	N	110	n
15	SI	(Shift In)	47	/	79	O	111	o
16	DLE	(Data link escape)	48	0	80	P	112	p
17	DC1	(Device control 1)	49	1	81	Q	113	q
18	DC2	(Device control 2)	50	2	82	R	114	r
19	DC3	(Device control 3)	51	3	83	S	115	s
20	DC4	(Device control 4)	52	4	84	T	116	t
21	NAK	(Negative acknowl.)	53	5	85	U	117	u
22	SYN	(Synchronous idle)	54	6	86	V	118	v
23	ETB	(End of trans. block)	55	7	87	W	119	w
24	CAN	(Cancel)	56	8	88	X	120	x
25	EM	(End of medium)	57	9	89	Y	121	y
26	SUB	(Substitute)	58	:	90	Z	122	z
27	ESC	(Escape)	59	;	91	[	123	{
28	FS	(File separator)	60	<	92	\	124	
29	GS	(Group separator)	61	=	93	]	125	}
30	RS	(Record separator)	62	>	94	^	126	~
31	US	(Unit separator)	63	?	95	_		
127	DEL	(Delete)						

Strings: Declaração e Inicialização

Declaração

A declaração de uma *string* é um simples vetor unidimensional do tipo *char*.

```
char minha_string[tamanho_string]
```

Como nos vetores, também podemos omitir o tamanho na declaração.

```
char minha_string[ ]
```

Strings: Declaração e Inicialização

Inicialização



```
1  #include <stdio.h>
2
3  int main() {
4      // Forma 1: Usando string literal
5      char nome1[30] = "gustavo";
6      char nome2[] = "gustavo";
7      // Forma 2: Usando lista de caracteres
8      char nome3[30] = {'g', 'u', 's', 't', 'a', 'v', 'o'};
9      return 0;
10 }
```

Strings: Entrada e Saída de Strings

Função *scanf*

A função *scanf* permite fazer leitura de *strings* usando *%s*.

Em relação ao uso de *scanf* para armazenar string devemos observar duas coisas:

- A função *scanf* realiza a leitura até encontrar um espaço, depois encerra a leitura e coloca o caracter terminador `\0`.
- A variável que vai armazenar a string não necessita ser precedida por `&`.

Por conta do `\0`, sempre precisamos reservar **mais do que o tamanho necessário**.

Strings: Entrada e Saída de Strings

The diagram shows the code `minha_string[tamanho+1]` where `tamanho+1` is highlighted in blue and enclosed in a red rounded rectangle. A red arrow points from this rectangle down to the text `\0`, which is also in red. This illustrates that the character at the index immediately following the current string length is the null terminator.

```
minha_string[tamanho+1]
```

`\0`

Ou seja, se o tamanho máximo de uma string é 100, precisamos reserva um espaço de $100 + 1 = 101$.

Strings: Entrada e Saída de Strings

Função *scanf*



```
1  #include <stdio.h>
2
3  int main() {
4      char nome[100];
5      printf("Digite o seu nome: ");
6      scanf("%s", nome);
7
8      printf("Oi, %s\n", nome);
9      return 0;
10 }
```

Strings: Entrada e Saída de Strings

Função *printf*



```
1  #include <stdio.h>
2
3  int main() {
4      char texto[] = "A aula está muito massa!";
5      printf("Uma verdade absoluta: %s\n", texto);
6      return 0;
7  }
```

Strings: Funções da Biblioteca <string.h>

A biblioteca <string.h> da linguagem C, contém uma série de funções para manipular strings.

Nesta seção veremos como:

- Copiar strings usando `strcpy()`;
- Concatenar strings usando `strcat()`;
- Descobrir o tamanho de uma string `strlen()`;
- Comparar strings em C usando `strcmp()`;

Há diversas outras funções disponíveis na biblioteca... Fica a sugestão pesquisar mais sobre 🤓

Strings: Funções da Biblioteca <string.h>

Em outras linguagens, operações de cópia/atribuição, concatenação e comparação de strings são operações simples em outras linguagens.

Em C, elas **não** são possíveis de serem realizadas diretamente, ou seja, é necessário utilizar as funções mencionadas no *slide* anterior.



Errar isso é muito comum :(

Strings: Funções da Biblioteca <string.h>



```
1  #include <stdio.h>
2
3  int main() {
4      char nome[] = "Gustavo";
5      char sobrenome[] = "Aragão";
6      // Não podemos atribuir/copiar strings dessa forma
7      nome = "Henrique";
8      // Operador + não suporta concatenação
9      char nome_concatenado[] = nome + sobrenome;
10     // Os operadores >, <, >=, <= e == não suporta comparação de strings
11     if (nome > sobrenome)
12         printf("nome > sobrenome\n");
13     else
14         printf("nome <= sobrenome\n");
15     return 0;
16 }
```

- strcpy(**string_destino**, **string_origem**)

Realiza a cópia do conteúdo da **string_origem** para a variável **string_destino**.

O parâmetro **string_origem** pode ser uma outra variável ou um String Literal.



```
1  int main() {
2      char nome[100] = "Gustavo";
3
4      // String Literal
5      nome = "Henrique";          // Inválido!
6      strcpy(nome, "Henrique");  // Válido!
7
8      // Outra variável
9      char outro_nome[] = "Henrique";
10     nome = outro_nome;          // Inválido!
11     strcpy(nome, outro_nome);  // Válido!
12
13     return 0;
14 }
```

- `strcat(string_destino, string_origem)`

Realiza a concatenação do conteúdo da `string_origem` com o conteúdo da variável `string_destino`.

O parâmetro `string_origem` pode ser uma outra variável ou um String Literal.

```
1 int main() {
2     char nome[100] = "Gustavo";
3     char outro_nome[100] = "Henrique";
4
5     // String Literal
6     nome = nome + "Henrique"; // Inválido!
7     strcat(nome, "Henrique"); // Válido!
8
9
10    // // Outra variável
11    nome = nome + outro_nome; // Inválido!
12    strcat(nome, outro_nome); // Válido!
13
14    return 0;
15 }
```

- strcmp(string1, string2)

Realiza a comparação do conteúdo de duas *strings*. Caso o retorno seja:

- == 0 → O conteúdo das duas strings são IGUAIS.
- < 0 → O conteúdo de string1 é MENOR LEXICOGRAFICAMENTE que o de string2.
- > 0 → O conteúdo de string1 é MAIOR LEXICOGRAFICAMENTE que o de string2.

Os parâmetros string1 e string2 podem ser uma variável ou um String Literal.

⚠ Lexicograficamente = Ordem alfabética, ou seja, como as palavras aparecem no dicionário.

```
1 int main() {
2     char nome[100] = "Gustavo";
3     char outro_nome[100] = "Henrique";
4
5     // Válido
6     if (nome == outro_nome) {
7         printf("nome == outro_nome\n");
8     }
9
10    // Inválido
11    if (strcmp(nome, outro_nome)) {
12        printf("nome == outro_nome\n");
13    }
14
15    return 0;
16 }
```

```
1 int main() {
2     char nome[100] = "Gustavo";
3     char outro_nome[100] = "Henrique";
4
5     // Literal
6     if (strcmp(nome, "Henrique") == 0) {
7         printf("nome == Henrique\n");
8     } else if (strcmp(nome, "Henrique") < 0) {
9         printf("nome < Henrique\n");
10    } else if (strcmp(nome, "Henrique") > 0) {
11        printf("nome > Henrique\n");
12    }
13
14    // Variável
15    if (strcmp(nome, outro_nome) == 0) {
16        printf("nome == outro_nome\n");
17    } else if (strcmp(nome, outro_nome) < 0) {
18        printf("nome < outro_nome\n");
19    } else if (strcmp(nome, outro_nome) > 0) {
20        printf("nome > outro_nome\n");
21    }
22
23    return 0;
24 }
```

- `strlen(string1)`

Descobre o tamanho do conteúdo da `string1`. A contagem é feita até encontrar o primeiro `\0`.

O parâmetro `string1` pode ser uma variável ou um String Literal.



```
1  int main() {
2      char nome[100] = "Gustavo";
3
4      // Variável
5      printf("%d\n", strlen(nome));
6      // String Literal
7      printf("%d\n", strlen("Gustavo"));
8
9      return 0;
10 }
```

Exercício 1: Operações com String

Escreva um programa em C que realize as seguintes operações com *strings*:

1. Ler o primeiro nome de uma pessoa (usando `scanf`).
2. Mostrar o tamanho do nome usando `strlen`.
3. Comparar se o nome digitado é igual a "Gustavo", mostrando o resultado da comparação.
4. Concatenar uma mensagem personalizada (ex: "Olá, <nome>!").

Exercício 1: Operações com String



```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char nome[100];
6
7      // Questão 1
8      printf("Digite o primeiro nome: ");
9      scanf("%s", nome);
10     printf("Nome = %s\n", nome);
11     // Questão 2
12     int tamanho_nome = strlen(nome);
13     printf("Tamanho = %d\n", tamanho_nome);
```



```
1      // Questão 3
2      char mensagem[110] = "Olá, ";
3      strcat(mensagem, nome);
4      strcat(mensagem, "!");
5      printf("Mensagem: %s\n", mensagem);
6      // Questão 4
7      if (strcmp(nome, "Gustavo") == 0) {
8          printf("nome == 'Gustavo'\n");
9      } else if (strcmp(nome, "Gustavo") < 0) {
10         printf("nome < 'Gustavo'\n");
11     } else if (strcmp(nome, "Gustavo") > 0) {
12         printf("nome > 'Gustavo'\n");
13     }
14     return 0;
15 }
```


Segunda Atividade Prática

Crie um programa em C que:

1. Leia uma **string** informada pelo usuário.
2. Verifique se a string é **palíndroma**.



Uma **palíndroma (ou palíndromo)** é uma palavra, frase ou número que pode ser lido da mesma forma **de trás para frente**, mantendo o mesmo sentido em ambas as direções.

Exemplos de palíndromos:

- arara
- reviver

Struct



Agradecimento a Lucas
Calumbi pelos slides!

O que são structs?

- Structs são um tipo de dado que permitem agrupar diferentes variáveis sob um mesmo nome.
- Elas são usadas para representar um conjunto de dados relacionados, como as informações de uma pessoa (nome, idade, altura).
- Cada variável dentro de uma struct é chamada de 'campo', e podem ter tipos de dados diferentes.



Como declarar uma struct?



```
struct pessoa
{
    char nome[51];
    int idade;
    float altura;
};

int main(void)
{
    struct pessoa pessoa_1;

    return 0;
}
```

Outra forma de
declarar struct:



```
typedef struct
{
    char nome[51];
    int idade;
    float altura;
} Pessoa;

int main(void)
{
    Pessoa pessoa_1;

    return 0;
}
```

Como declarar e
inicializar uma
struct?



```
typedef struct
{
    char nome[51];
    int idade;
    float altura;
} Pessoa;

int main(void)
{
    Pessoa pessoa_1 = {"Davi Fontes", 21, 1.75};

    return 0;
}
```

Como declarar um vetor de struct



```
typedef struct
{
    char nome[51];
    int idade;
    float altura;
} Pessoa;

int main(void)
{
    Pessoa vetor[5];
    vetor[0].idade = 30;
    vetor[2].altura = 1.45;

    return 0;
}
```

Como manipular uma struct?



Manipulando campos da struct



```
int main(void)
{
    Pessoa pessoa1 = {"Irandi Silva", 20, 1.85};

    mostre_informacoes(pessoa1);

    strcpy(pessoa1.nome, "Rodrigo Faro");
    pessoa1.idade = 51;
    pessoa1.altura = 1.85;

    mostre_informacoes(pessoa1);

    return 0;
}
```

```

#include <stdio.h>
#include <string.h>

```

```

typedef struct
{
    char nome[50];
    int idade;
    float altura;
} Pessoa;

```

```

void mostre_informacoes(Pessoa pessoa)
{
    printf("Nome: %s\n", pessoa.nome);
    printf("Idade: %d\n", pessoa.idade);
    printf("Altura: %.2f\n\n", pessoa.altura);
}

```

```

int main(void)
{
    Pessoa pessoal = {"Irandi Silva", 20, 1.85};

    mostre_informacoes(pessoal);

    strcpy(pessoal.nome, "Rodrigo Faro");
    pessoal.idade = 51;
    pessoal.altura = 1.85;

    mostre_informacoes(pessoal);

    return 0;
}

```



Como parâmetro de uma função



Alterando o valor de um campo

Atividade prática para casa:

Utilizando a Struct definida na atividade anterior, crie um programa em C para cadastrar informações de três alunos e exibir suas informações na tela.

- Peça ao usuário para inserir os dados de três alunos
- Armazene os dados em um array de alunos
- Após o cadastro, exiba as informações de todos os alunos
- Use 'for' para evitar repetições de código

Uma solução possível:

```
int main()
{
    Aluno alunos[3];

    for (int i = 0; i < 3; i++) {
        printf("Digite o nome do aluno %d: ", i + 1);
        scanf(" %s", alunos[i].nome);

        printf("Digite a matrícula do aluno %d: ", i + 1);
        scanf("%d", &alunos[i].matricula);

        printf("Digite a nota do aluno %d: ", i + 1);
        scanf("%f", &alunos[i].nota);

        printf("\n");
    }

    printf("Informações dos Alunos:\n");
    for (int i = 0; i < 3; i++) {
        printf("Aluno %d:\n", i + 1);
        printf("Nome: %s\n", alunos[i].nome);
        printf("Matrícula: %d\n", alunos[i].matricula);
        printf("Nota: %.2f\n\n", alunos[i].nota);
    }

    return 0;
}
```

```
#include <stdio.h>
#include <string.h>

typedef struct
{
    char nome[50];
    int matricula;
    float nota;
} Aluno;
```

Depuração



Depuração: O que é e Cadê o erro?

A Arte de Caçar Bugs: Encontrar, isolar e corrigir anomalias no código.

Tipos de Erros:

- *Sintaxe / Compilação*: O compilador te avisa (Ex: faltou ;).
- *Execução (Runtime)*: O programa "crasha" do nada (Ex: divisão por zero).
- *Lógicos*: O programa roda, mas o resultado é bizarro.

O Foco: Onde a sintaxe está certa, mas a lógica falhou.

Depuração: Ferramentas de Depuração

- GDB (GNU Debugger): O Detetive. Analisa o código passo a passo.
- Valgrind: O Fiscal da Memória. Encontra vazamentos e acessos indevidos.
- gprof (GNU Profiler): O Cronometrista. Descobre onde o código está perdendo tempo.

Regra de Ouro: Para a magia funcionar, compile com a **flag -g**
(Ex: **gcc -g programa.c -o programa**).

Depuração: GDB (GNU Debugger)

- Quando usar?
 - O programa entra em loop infinito, variáveis têm valores absurdos ou ocorre o famoso *Segmentation Fault*.
- O que ele faz?
 - Pausa a execução do programa na linha que você quiser.
- Como usar?
 - `gdb ./programa`

Depuração: GDB (GNU Debugger)

Command	Short Form	Description	Example
break	b	Set breakpoint	b main, b 15, b function_name
run	r	Start program execution	r arg1 arg2
continue	c	Continue execution	c
step	s	Step into functions	s
next	n	Step over functions	n
print	p	Print variable value	p variable_name
backtrace	bt	Show call stack	bt
list	l	Show source code	l, l 10, l function_name
info	i	Show information	i registers, i breakpoints
quit	q	Exit GDB	q

```
1  #include <stdio.h>
2  #include <string.h>
3
4
5  struct Aluno {
6      int matricula;
7      char *nome; // Ponteiro solto, não alocou memória!
8  };
9
10 int main() {
11     struct Aluno a1;
12     a1.matricula = 101;
13
14     printf("Registrando aluno...\n");
15
16     // O programa vai dar SEGFAULT exatamente aqui:
17     strcpy(a1.nome, "Maria");
18
19     printf("Aluno: %s\n", a1.nome);
20     return 0;
21 }
```

```
(gdb) run
```

```
Starting program: /home/raiz/Estudo - Especifico/C/lista4/a
```

```
This GDB supports auto-downloading debuginfo from the following URLs:
```

```
<ima:enforcing>
```

```
<https://debuginfod.fedoraproject.org/>
```

```
<ima:ignore>
```

```
Enable debuginfod for this session? (y or [n]) y
```

```
Debuginfod has been enabled.
```

```
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
```

```
[Thread debugging using libthread_db enabled]
```

```
Using host libthread_db library "/lib64/libthread_db.so.1".
```

```
Registrando aluno...
```

```
Program received signal SIGSEGV, Segmentation fault.
```

```
main () at a.c:17
```

```
17         strcpy(a1.nome, "Maria");
```

```
(gdb) █
```

Depuração: Valgrind

- Quando usar?
 - Suspeita de *Memory Leak* (vazamento de memória) ou uso de ponteiros não inicializados
- O que ele faz?
 - Monitora cada **malloc** e **free** do seu código
- Como rodar?
 - O comando básico monitora vazamentos de memória e erros de ponteiro:

```
valgrind ./programa
```

Depuração: Valgrind

- Para obter um relatório detalhado e completo sobre vazamentos de memória, use estas flags adicionais:

```
valgrind --leak-check=full --show-leak-kinds=all ./programa
```

- O que buscar:
 - All heap blocks were freed -- no leaks are possible
- Não possui comandos internos, em vez disso controla o comportamento dele usando argumentos e flags no momento da execução.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  struct Livro {
6      char *titulo;
7      int paginas;
8  };
9
10 int main() {
11     // Alocando memória para a struct e para a string
12     struct Livro *meuLivro = malloc(sizeof(struct Livro));
13     meuLivro->titulo = malloc(50 * sizeof(char));
14
15     strcpy(meuLivro->titulo, "Aprendendo C");
16     meuLivro->paginas = 250;
17
18     printf("Lendo: %s\n", meuLivro->titulo);
19
20
21     return 0;
22 }
23
```

```
● raiz@fedora:~/Estudo - Especifico/C/lista4$ gcc a.c -o a
● raiz@fedora:~/Estudo - Especifico/C/lista4$ valgrind --leak-check=full ./a
==106282== Memcheck, a memory error detector
==106282== Copyright (C) 2002-2026, and GNU GPL'd, by Julian Seward et al.
==106282== Using Valgrind-3.27.1 and LibVEX; rerun with -h for copyright info
==106282== Command: ./a
==106282==
Lendo: Aprendendo C
==106282==
==106282== HEAP SUMMARY:
==106282==    in use at exit: 66 bytes in 2 blocks
==106282==   total heap usage: 3 allocs, 1 frees, 1,090 bytes allocated
==106282==
==106282== 66 (16 direct, 50 indirect) bytes in 1 blocks are definitely lost in loss record 2 of 2
==106282==    at 0x4840AE6: malloc (vg_replace_malloc.c:447)
==106282==    by 0x400487: main (in /home/raiz/Estudo - Especifico/C/lista4/a)
==106282==
==106282== LEAK SUMMARY:
==106282==    definitely lost: 16 bytes in 1 blocks
==106282==    indirectly lost: 50 bytes in 1 blocks
==106282==    possibly lost: 0 bytes in 0 blocks
==106282==    still reachable: 0 bytes in 0 blocks
==106282==    suppressed: 0 bytes in 0 blocks
==106282==
==106282== For lists of detected and suppressed errors, rerun with: -s
==106282== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Depuração: Gprof (GNU Profiler)

- Quando usar?
 - O código está muito lento e você quer otimizar (ótimo para maratonas de programação).
- O que ele faz?
 - Mostra quanto tempo (%) o programa passou dentro de cada função.
- Como usar:
 1. Compilar com -pg: `gcc -pg programa.c -o prog`
 2. Rodar: `./prog` (Gera um arquivo `gmon.out`)
 3. Analisar: `gprof ./prog gmon.out`

O Resultado: Uma tabela clara apontando a função mais "pesada" do código.

Desafios



Desafio 1: Contagem de Letras

Dada uma palavra composta apenas por letras minúsculas (de a até z), escreva um programa em C que conte quantas vezes cada letra aparece.

 Não é permitido criar uma variável para cada letra. Você deve usar um vetor para contar as letras.

- **Entrada**

Uma string (palavra) com até **100 caracteres**.

- **Saída**

Para cada letra que aparecer na palavra, imprimir no formato:

<letra>: quantidade

A saída deve estar em **ORDEM ALFABÉTICA**.

Desafio 1: Contagem de Letras

- **Exemplo:**
 - **Entrada:**

banana

- **Saída:**

a: 3

b: 1

n: 2

Desafio 1: Contagem de Letras

LETRAS

Vetor de Contagem

'a'

`cnt['a'] = 0`

'b'

`cnt['b'] = 0`

'c'

`cnt['c'] = 0`

'd'

`cnt['d'] = 0`

...

...

'z'

`cnt['z'] = 0`

Desafio 1: Contagem de Letras

banana

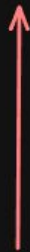
`cnt['a'] = 0`

`cnt['b'] = 0`

`cnt['n'] = 0`

Desafio 1: Contagem de Letras

banana



`cnt['a'] = 0`

`cnt['b'] = 1`

`cnt['n'] = 0`

Desafio 1: Contagem de Letras

banana



`cnt['a'] = 1`

`cnt['b'] = 1`

`cnt['n'] = 0`

Desafio 1: Contagem de Letras

banana



`cnt['a'] = 1`

`cnt['b'] = 1`

`cnt['n'] = 1`

Desafio 1: Contagem de Letras

banana



`cnt['a'] = 2`

`cnt['b'] = 1`

`cnt['n'] = 1`

Desafio 1: Contagem de Letras

banana



`cnt['a'] = 3`

`cnt['b'] = 1`

`cnt['n'] = 2`

Desafio 1: Contagem de Letras

Implementação

Vetor de Contagem

cnt['a'] = 0

cnt['b'] = 0

cnt['c'] = 0

cnt['d'] = 0

...

cnt['z'] = 0

índice = c - 'a'

int cnt[26]

cnt[0] = 0

cnt[1] = 0

cnt[2] = 0

cnt[3] = 0

...

cnt[25] = 0

Obrigado pela atenção

